

P4.1. Inici (copy)

1. Análisis del ciclo de vida de una petición asíncrona

Cuando JavaScript hace una petición AJAX al servidor, puede hacerlo de forma síncrona o asíncrona.

Si es **síncrona**, el programa se queda esperando la respuesta y no sigue ejecutando el resto del código. Eso bloquea la página.

Si es **asíncrona**, la petición se hace en segundo plano y el código puede seguir ejecutándose normalmente. Mientras tanto, el navegador se encarga de enviar la petición y esperar la respuesta del servidor.

Cuando la respuesta llega, se ejecuta una **callback**, que es una función que se había dejado preparada para ese momento. Sirve para procesar los datos recibidos, por ejemplo mostrándolos en la página.

2. Anatomía y configuración del objeto XMLHttpRequest

Para usar XMLHttpRequest, primero se crea el objeto y luego se configura con `open(metodo, url, async)`.

- **metodo**: tipo de petición, como GET o POST.
- **url**: dirección a la que se hace la petición.
- **async**: indica si será asíncrona (`true`) o síncrona (`false`).

Después se usa `send()` para enviar la petición.

En un GET normalmente va vacío, y en un POST se le pasan los datos que se quieren enviar.

3. Comunicación bidireccional: GET y POST

La petición **GET** se usa para leer datos del servidor.

La petición **POST** se usa para enviar datos.

En una petición POST normalmente hay que añadir una cabecera, y los datos se suelen enviar en el cuerpo de la petición.

En una petición GET, en cambio, la cabecera solo es necesaria en algunos casos, como por ejemplo para autenticación.